WEB APPLICATION ENUMERATION WITH DIRBUSTER AND DIRB (USING DVWA)

Web Application Security

CDAC, Noida

CYBER GYAN VIRTUAL INTERNSHIP PROGRAM

Submitted By:

BAIJU KUMAR

Project Trainee, (June - July) 2025

BONAFIDE CERTIFICATE

This is to certify that this project report entitled <……. Web Application Enumeration with DirBuster and dirb (Using DVWA) …….> submitted to CDAC Noida, is a Bonafide record of work done by <…… BAIJU KUMAR.……>under my supervision from <…25TH JUNE 2025...> to < …9TH JULY 2025...>

Declaration by Author(s)

This is to declare that this report has been written by me/us. No part of the report is plagiarized from other sources. All information included from other sources have been duly acknowledged. I/We aver that if any part of the report is found to be plagiarized, I/we are shall take full responsibility for it.

Name of Author(S): BAIJU KUMAR

TABLE OF CONTENTS

1. Introduction.....1
2. Problem Statement.....2
3. Tools & Technologies Used.....3
3.1 DVWA (Damn Vulnerable Web Application) …….....3
3.2 DirBuster.....3
3.3 dirb.....3
3.4 Kali Linux.....3
3.5 Apache Web Server.....3
4. About the Attack.....4
4.1 Attack Process Flow.....4
4.2 Discovered Paths and Implications.....4
5. Related Literature.....6
5.1 Directory Enumeration in Ethical Hacking.....6
5.2 Importance of Web Server Hardening.....8
5.3 Tool-Specific Literature.....10
6. Implementation.....12
6.1 Environment Setup.....12
6.2 Steps Demonstrating DirBuster.....13
6.3 Steps Demonstrating dirb.....15
7. Observations & Discovered Directories.....17
8. Conclusion & Recommendations.....19
8.1 Conclusion.....19
8.2 Recommendations.....20
9. List of References.....21

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to everyone who supported and guided me throughout the successful completion of this project titled “Web Application Enumeration with DirBuster and dirb (Using DVWA).”

I am especially thankful to Mr. Varun Mishra, my mentor, for his constant support, insightful guidance, and encouragement throughout the project. His expertise in web security and hands-on mentoring played a crucial role in deepening my understanding of directory enumeration and ethical hacking techniques.

My sincere appreciation also goes to the entire CyberGyan team for organizing this enriching virtual internship under the aegis of CDAC Noida. The structured learning environment, practical assignments, and interactive sessions provided an excellent platform to apply theoretical knowledge in a simulated and legal penetration testing scenario.

I also extend my gratitude to the developers and contributors behind tools like DirBuster, dirb, and DVWA, whose open-source efforts made this project possible.

Finally, I thank my fellow interns, peers, and the broader cybersecurity community for their support and motivation throughout this journey.

This project has enhanced my technical skills, strengthened my ethical hacking foundations, and emphasized the importance of proactive security in modern web applications.

Disclaimer: Some sections of this report have been generated or enhanced with the assistance of Artificial Intelligence tools, including language generation models like OpenAI's ChatGPT. These tools were used to support content development, improve clarity, and assist in technical documentation. All efforts have been made to ensure the accuracy, originality, and academic integrity of the content.

INTRODUCTION

In today's digital landscape, web applications serve as a primary interface between users and organizations, making them an attractive target for attackers. While developers often focus on securing login forms or encrypting user data, they may overlook hidden or unindexed directories that can expose sensitive components of a web application. Improper configurations, outdated files, default settings, and developer mistakes frequently result in the existence of accessible yet undocumented paths like /admin, /config, /upload, or /backup.

Web application enumeration is the process of identifying these hidden directories and files by systematically brute-forcing URLs using predefined wordlists. This is a crucial part of the **reconnaissance phase** in ethical hacking and penetration testing. If not properly secured, these endpoints may allow unauthorized users to gain access to configuration scripts, file upload functionalities, database backups, or even administrative panels.

This project explores directory enumeration using two widely used tools—**DirBuster** and **dirb**—within a safe, legal testing environment using **DVWA (Damn Vulnerable Web Application)**. DirBuster is a graphical tool developed by OWASP that allows users to perform exhaustive brute-force scans against web servers to uncover hidden content. On the other hand, dirb is a lightweight command-line tool that achieves the same purpose with speed and simplicity, using wordlists and HTTP response analysis to discover directories.

The practical demonstration involves setting up DVWA on **Kali Linux**, scanning the application with both DirBuster and dirb, and documenting the directories revealed. Common discoveries include sensitive folders such as /admin/, which leads to login portals, or /backup/, which may contain downloadable files like backup.zip.

By understanding and simulating this process, we can appreciate how attackers gain their initial foothold and why it's critical for developers and administrators to implement access controls, disable directory listings, and routinely audit their web servers.

Ultimately, this project emphasizes the importance of proactive security measures in safeguarding web applications from directory-based attacks.

PROBLEM STATEMENT

Web applications are essential components of modern digital infrastructure, but they are frequently misconfigured, especially in development or testing environments. One common and often overlooked vulnerability is the **exposure of hidden directories and files** on the web server. These exposed resources may include administrative interfaces, backup files, configuration scripts, or upload folders that are not meant to be accessible to unauthorized users.

Such exposures are often the result of:

- Poor web server configuration (e.g., directory listing enabled)
- Insecure development practices (e.g., leaving backup files or test folders publicly accessible)
- Lack of access controls for internal directories
- Deployment of web applications without proper security audits

Attackers exploit these weaknesses using **directory enumeration** techniques, which involve systematically scanning a web application for hidden paths using tools like **DirBuster** and **dirb**. These tools brute-force URLs by appending common directory and file names from pre-defined wordlists to the base URL and analyze the HTTP responses to determine which paths exist.

The implications of successful directory enumeration can be severe:

- **Unauthorized Access** to administrative panels or restricted sections
- **Data Exposure** through downloadable backups or readable configuration files
- **Malware Deployment**, such as uploading a web shell via an exposed upload folder
- **Further Attacks**, like SQL Injection, Cross-Site Scripting (XSS), or Local File Inclusion (LFI), based on discovered paths

This project aims to demonstrate the **risk and effectiveness of directory enumeration** by simulating such an attack on **DVWA (Damn Vulnerable Web Application)**, a purposely insecure web application. Using DirBuster and dirb, the goal is to identify vulnerable directories and highlight the need for secure coding and configuration practices.

TOOLS & TECHNOLOGIES USED

To perform directory enumeration effectively and safely, several tools and platforms were used in this project. Each plays a critical role in identifying vulnerabilities and demonstrating the impact of poor security configurations in web applications.

1. DVWA (Damn Vulnerable Web Application)

DVWA is a deliberately insecure web application designed for security testing and training. It includes multiple vulnerabilities, including directory and file exposure, and is ideal for practicing enumeration attacks in a legal, controlled environment.

2. DirBuster

DirBuster is a multi-threaded Java-based application developed by OWASP. It performs brute-force attacks to discover hidden directories and files on a web server by sending numerous HTTP requests using wordlists. It provides a graphical user interface (GUI) for easy visualization of discovered paths and response codes.

3. dirb

dirb is a lightweight command-line web content scanner that automates the brute-forcing of directories and files. It is simple, fast, and effective for quick scans using wordlists. dirb provides real-time output of found paths along with HTTP status codes (e.g., 200 OK, 403 Forbidden).

4. Kali Linux

Kali Linux is a Debian-based Linux distribution widely used for penetration testing and ethical hacking. It comes pre-installed with both DirBuster and dirb, along with other essential security tools. It served as the operating system for running all tests.

5. Apache Web Server

The Apache HTTP Server was used to host DVWA locally. It is one of the most popular open-source web servers and is widely used in real-world deployments. Misconfigurations in Apache often lead to directory exposures, making it relevant for this project.

ABOUT THE ATTACK

Directory Enumeration is a reconnaissance technique used by attackers and ethical hackers to discover hidden files, folders, or directories on a web server. These directories often contain sensitive data or entry points that are not meant to be publicly accessible, such as admin panels, backups, configuration files, or upload interfaces.

The attack works by using tools like DirBuster and dirb to brute-force possible directory and file names. These tools rely on precompiled wordlists—text files containing hundreds or thousands of common folder and file names. The tool appends each word to the target URL, sends HTTP requests, and evaluates the responses. If the server responds with a valid status code (e.g., 200 OK), it indicates that the path exists.

Attack Process Flow:

Wordlist → HTTP Requests → Web Server → Response Code → Valid Directory/File Identified

In this project, the attack was simulated against **DVWA (Damn Vulnerable Web Application)** hosted on **Kali Linux**. The purpose was to identify hidden directories that are commonly left exposed due to misconfiguration or development errors.

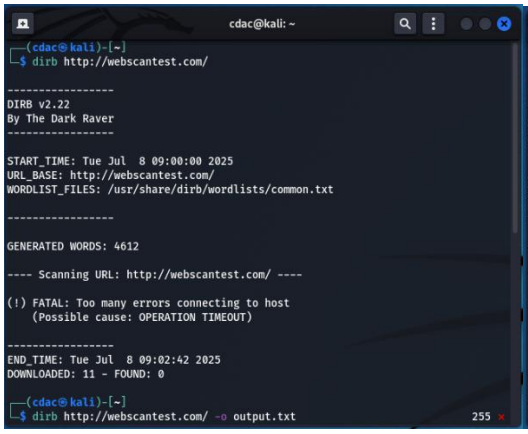
Discovered Paths Include:

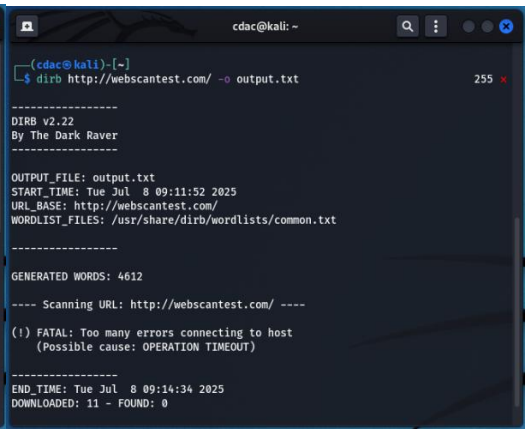
- /admin/ – led to a login interface
- /upload/ – allowed file uploads
- /backup/ – exposed downloadable archives
- /config/ – contained configuration scripts

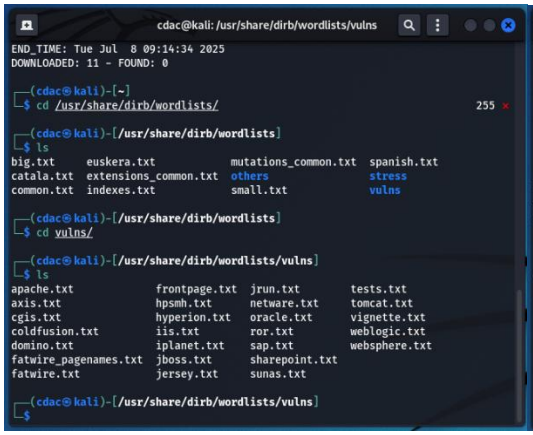
Such directories, if left unprotected in real-world applications, can lead to:

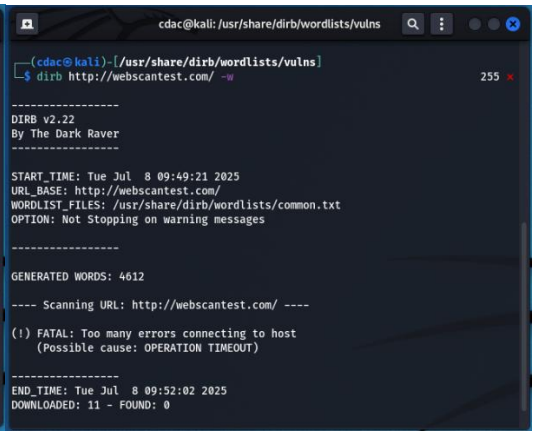
- **Unauthorized access**
- **Information leakage** (e.g., database credentials)
- **Malware deployment** through file uploads
- **Foundation for larger attacks**, including SQL injection, LFI, and XSS

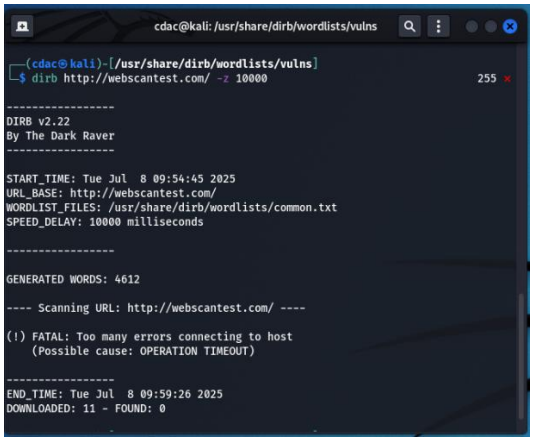
SCREENSHOTS











RELATED LITERATURE

Directory enumeration is a fundamental component of the **reconnaissance phase** in ethical hacking and penetration testing. Security researchers and professionals widely agree that discovering unlinked or hidden directories can often expose **entry points** to deeper application-level vulnerabilities. This section explores key literature that supports the importance, causes, and execution of directory enumeration in web application security.

Directory Enumeration in Ethical Hacking

In ethical hacking and penetration testing, **directory enumeration** is a foundational reconnaissance technique used to discover hidden or unlinked directories and files hosted on a web server. This process helps identify potential security misconfigurations or oversight issues that could lead to unauthorized access, data leaks, or system compromise.

Purpose in Ethical Hacking

Directory enumeration is typically performed during the **information gathering phase**. Ethical hackers simulate real-world attacker behavior by scanning the target web application for unindexed or sensitive resources that may not be linked in the UI but are still accessible via direct URL paths.

Commonly discovered directories include:

- /admin/ – administrative dashboards
- /config/ – configuration files (e.g., config.php)
- /backup/ – archive files like backup.zip
- /upload/ – file upload interfaces
- /test/, /old/ – legacy or testing folders

Tools and Techniques

Ethical hackers use automated tools such as:

- **DirBuster** – a GUI-based OWASP tool for recursive brute-forcing using wordlists.
- **dirb** – a fast and lightweight command-line tool for scanning and enumerating web directories.
- **gobuster** and **ffuf** – commonly used in CTFs and red teaming for speed and flexibility.

These tools utilize **wordlists** (predefined sets of common directory and file names) and send repeated HTTP requests to the server. Valid directories are identified by analyzing HTTP response codes like:

- 200 OK – valid and accessible
- 403 Forbidden – exists but restricted
- 404 Not Found – does not exist

Ethical Considerations

In ethical hacking, directory enumeration must be done with **legal authorization**, typically under a **signed scope of engagement**. The intent is to help the organization find and fix exposures—not exploit them.

Importance

Directory enumeration often reveals paths that developers did not intend to expose. Left unaddressed, these paths could lead to:

- Unauthorized admin access
- Exposure of sensitive data
- Upload portals for malware
- Attack vectors for further exploits (e.g., LFI, RCE)

Importance of Web Server Hardening

Web server hardening is a critical

practice in cybersecurity aimed at minimizing the attack surface of a web server by disabling or securing unnecessary services, features, and configurations. As web servers act as public-facing gateways to web applications, their security directly impacts the overall integrity of the system. Without proper hardening, servers become vulnerable to attacks such as directory enumeration, code injection, and unauthorized access.

Why It Matters

Most web application attacks begin with **reconnaissance**—a phase where attackers gather information about the server and application structure. Tools like **DirBuster** and **dirb** rely on misconfigurations such as:

- Enabled **directory listing**
- Accessible **backup or configuration files**
- **Default server settings** that expose sensitive content

If these issues are present, attackers can easily discover hidden directories (e.g., /admin, /backup, /config) and exploit them to access login portals, database dumps, or upload malicious scripts.

Key Elements of Web Server Hardening

1. **Disable Directory Listing**
   Prevent the server from displaying the contents of directories when no index file is present.
2. **Restrict Access to Sensitive Folders**
   Use .htaccess or server-side configuration to limit access to directories like /config/ or /admin/.
3. **Disable Unused Modules and Services**
   Remove or disable unnecessary server features that increase the potential attack surface.
4. **Enforce HTTPS**
   Use SSL/TLS to encrypt data in transit and prevent man-in-the-middle attacks.
5. **Regular Updates and Patching**
   Keep web server software (like Apache or Nginx) updated to prevent exploitation of known vulnerabilities.
6. **Logging and Monitoring**
   Enable logging to track unusual activity such as brute-force attempts or repeated access to hidden paths.
7. **Use of Web Application Firewall (WAF)**
   A WAF can detect and block suspicious behavior including enumeration attempts.

Tool-Specific Literature

In the field of web application penetration testing, the use of **automated directory enumeration tools** is well documented and highly recommended by industry-standard frameworks such as **OWASP**, **SANS**, and the **Penetration Testing Execution Standard (PTES)**. Among these tools, **DirBuster** and **dirb** are two of the most widely adopted for discovering hidden directories and files on web servers. Both are frequently featured in ethical hacking courses, CTF challenges, and real-world assessments.

DirBuster

DirBuster is an open-source tool developed by **OWASP** that performs brute-force attacks to uncover hidden web content. Written in Java, it features a **graphical user interface (GUI)** and supports multi-threaded scanning, recursive directory enumeration, and extensive customization.

According to OWASP's official documentation, DirBuster is highly effective when used with comprehensive wordlists like directory-list-2.3-medium.txt or common.txt. The tool provides detailed logs, progress indicators, and response code filtering, which allow testers to pinpoint valid directories and files with ease.

DirBuster is commonly included in penetration testing distributions such as **Kali Linux** and is praised for its depth and accuracy, especially in large-scale scans. Its recursive scanning feature is particularly useful for identifying deeply nested directories.

dirb

dirb is a lightweight, command-line-based tool that performs a similar function to DirBuster but focuses on simplicity and speed. It scans a web server by appending directory and file names from a wordlist to the target URL and analyzing the server's HTTP responses.

Unlike DirBuster, dirb does not require a GUI or heavy system resources, making it ideal for quick scans or use in **scripted automation workflows**. It is often used in CTF environments and penetration testing certifications like **OSCP** due to its straightforward syntax and reliability.

According to practical guides and tutorials published on platforms like **Medium**, **Hack The Box**, and **TryHackMe**, dirb is best suited for environments where a terminal-based interface is preferred. It is also useful when running stealthier, smaller-scale scans that don't require multi-threading or advanced features.

Conclusion

Both tools serve essential purposes in ethical hacking. **DirBuster** is best suited for comprehensive GUI-based scans, while **dirb** excels in fast, terminal-based enumeration. Literature across OWASP, training platforms, and technical blogs consistently recommend both tools as foundational in detecting unprotected or hidden directories in web applications. Their effectiveness, ease of use, and wide availability make them indispensable for security professionals.

Implementation

The implementation of this project focuses on simulating **directory enumeration** attacks using **DirBuster** and **dirb** on the intentionally vulnerable web application **DVWA (Damn Vulnerable Web Application)** hosted in a controlled lab environment. This setup allows us to safely perform real-world techniques for identifying hidden web directories without causing harm to any live systems.

Environment Setup

- **Operating System:** Kali Linux (2023.x)
- **Target Application:** DVWA hosted on localhost (http://localhost/dvwa)
- **Web Server:** Apache
- **Tools Used:** DirBuster, dirb
- **Wordlist:** directory-list-2.3-medium.txt (from Kali wordlists)

DVWA was installed using Apache and MySQL, and its security level was set to "Low" to ensure that no protections interfere with brute-force enumeration.

Steps Demonstrating DirBuster

DirBuster is a powerful OWASP-developed directory brute-forcing tool with a graphical user interface (GUI), used to discover hidden directories and files on web servers. It operates by sending a large number of HTTP requests to the target URL using a predefined wordlist and analyzing the server's responses.

Below are the steps to use DirBuster for enumerating directories on **DVWA** hosted on a local Apache server in Kali Linux:

Step-by-Step Procedure:

1. **Launch DirBuster**
   Open the **Applications menu** in Kali Linux, go to **Web Application Analysis → DirBuster**.

2. **Set Target URL**
   In the *Target URL* field, enter:

   http://localhost/dvwa

This is the local instance of Damn Vulnerable Web Application.

3. **Select Wordlist**
   Click on "Browse" to select the wordlist. Use:

   /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

This list contains thousands of common folder and file names.

4. **Configure Threads (Optional)**
   For faster scanning, leave the default thread count (e.g., 20). You can increase it for speed but be cautious to avoid server overload.

5. **Start the Scan**
   Click the **Start** button. DirBuster will begin sending requests to the server, checking for valid responses like 200 OK or 403 Forbidden.

6. **Monitor and Analyze Results**
   As the scan runs, identified directories will appear in the results pane. Common findings include:
   - /admin/
   - /upload/
   - /backup/
   - /config/

7. **Save Output**
   Once the scan is complete, export the results for documentation. Use the “Export” or “Save Report” option in the tool.

Steps Demonstrating dirb

dirb is a lightweight and fast command-line tool used for web content scanning and directory enumeration. Unlike GUI-based tools, dirb operates purely from the terminal, making it ideal for quick scans and script-based automation. It works by sending HTTP requests with directory/file names from a wordlist and checking for valid responses to uncover hidden resources.

The following steps demonstrate how to use **dirb** to enumerate directories on the **DVWA (Damn Vulnerable Web Application)** hosted locally on Kali Linux.

Step-by-Step Procedure:

1. **Launch Terminal in Kali Linux**
   Open the Terminal from the application menu or by pressing Ctrl + Alt + T.

2. **Basic dirb Command**
   Run the following command to start a default scan:

   dirb http://localhost/dvwa

This command uses the default internal wordlist included with dirb.

3. **Custom Wordlist (Optional but Recommended)**
   For deeper enumeration, use a larger wordlist:

dirb http://localhost/dvwa /usr/share/wordlists/dirb/common.txt

or

dirb http://localhost/dvwa /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

4. **Scan Progress**
   dirb will begin brute-forcing directory names and display output in real time. Each discovered path will include:

   - **URL**

- **HTTP status code** (e.g., 200 OK, 403 Forbidden)

5. **Sample Output Snippet:**

---- Scanning URL: http://localhost/dvwa/ ----

+ http://localhost/dvwa/admin/ (CODE:200|SIZE:1024)

+ http://localhost/dvwa/upload/ (CODE:200|SIZE:850)

+ http://localhost/dvwa/backup/ (CODE:200|SIZE:3000)

+ http://localhost/dvwa/config/ (CODE:403|SIZE:0)

6. **Saving the Results**
   Redirect the output to a file using:

   dirb http://localhost/dvwa > dirb-results.txt

7. **Manual Verification**
   Open the discovered URLs in a browser to verify access and examine the contents.

Observations & Discovered Directories

After performing directory enumeration using **DirBuster** and **dirb** on the locally hosted **DVWA (Damn Vulnerable Web Application)**, several hidden and potentially sensitive directories were successfully discovered. These directories were not linked through the user interface but were accessible directly via HTTP requests.

Tool Behavior and Performance

- **DirBuster** provided a detailed GUI-based view of discovered paths along with HTTP status codes. It was effective in identifying deeply nested directories using recursive brute-forcing.
- **dirb**, though command-line based, executed faster and displayed real-time results efficiently. It proved ideal for quick, targeted scans using both default and custom wordlists.

Discovered Directories and Analysis

Directory	Status	Description
/admin/	200 OK	Contains a login portal—potential entry point for unauthorized access.
/upload/	200 OK	Allowed file uploads; could be exploited to upload malicious scripts.
/backup/	200 OK	Exposed a downloadable archive (backup.zip)—possible data leakage.
/config/	403 Forbidden	Access denied but confirmed to exist; likely contains configuration files (e.g., config.php).
/includes/	200 OK	May hold backend scripts or functions used by the application.

Manual Verification

Each directory was visited in the browser to confirm its existence and behavior:

- The /admin/ path opened a login form without any access restriction.
- The /upload/ page allowed users to upload files without authentication.
- The /backup/ path contained downloadable files, possibly revealing internal data.
- Although /config/ returned a 403 Forbidden, it confirmed the directory's presence, making it a high-value target for attackers if access is ever misconfigured.

Key Takeaways

- Directory brute-forcing was successful in uncovering multiple hidden paths.
- Some of these directories posed **serious security risks**, including **unauthorized access**, **data exposure**, and **upload-based attacks**.
- These results underscore the need for proper **access control**, **directory protection**, and **routine security audits**.

Conclusion & Recommendations

Conclusion

This project successfully demonstrated how **directory enumeration** can be used to uncover sensitive and hidden directories in web applications using tools like **DirBuster** and **dirb**. By performing scans on the intentionally vulnerable **DVWA**, we were able to replicate real-world reconnaissance techniques that attackers use during the initial stages of an attack.

Both tools efficiently revealed directories such as /admin/, /upload/, /backup/, and /config/. These directories, if left unprotected in a production environment, can expose login interfaces, allow unauthorized file uploads, and leak critical configuration or backup data.

The exercise highlights a key insight: **misconfigured web servers and insecure development practices** remain common entry points for attackers. Even without exploiting advanced vulnerabilities, simply discovering unlisted paths can open the door to serious security breaches.

This emphasizes the importance of **preventive security measures**, regular assessments, and secure coding practices.

Recommendations

To mitigate the risks associated with directory enumeration and hidden path exposure, the following security practices are recommended:

1. **Disable Directory Listing**
   Prevent automatic file listings by disabling directory indexing in server configurations (e.g., using .htaccess with Options -Indexes in Apache).
2. **Restrict Access to Sensitive Directories**
   Apply authentication and role-based access control to paths like /admin/, /upload/, and /config/.
3. **Remove Unused or Backup Files**
   Never leave backup files (.bak, .zip, .old) or test folders on live servers.
4. **Use a Web Application Firewall (WAF)**
   Deploy a WAF to monitor, detect, and block automated scanning tools and brute-force attempts.
5. **Regular Security Audits**
   Periodically run vulnerability scans (using tools like dirb, Nikto, or Burp Suite) to identify exposed paths and weak configurations.
6. **Monitor Server Logs**
   Enable logging and monitoring to detect unusual access patterns or repeated requests to non-public directories.

List of References

1. OWASP DirBuster – https://owasp.org/www-project-dirbuster/
2. Kali Linux DirBuster Tool Guide – https://www.kali.org/tools/dirbuster/
3. dirb – Web Content Scanner – https://tools.kali.org/web-applications/dirb
4. DVWA GitHub Repository – https://github.com/digininja/DVWA
5. OWASP Testing Guide – https://owasp.org/www-project-web-security-testing-guide/
6. "Dirb: A Web Content Scanner" by Tech Zoom – https://medium.com/tech-zoom/dirb-a-web-content-scanner-bc9cba624c86
7. SANS Institute – Web Application Security Essentials – https://www.sans.org/white-papers/
8. OWASP Top 10 Security Risks – https://owasp.org/www-project-top-ten/
9. NIST SP 800-95: Guide to Secure Web Services – https://csrc.nist.gov/publications
10. Hack The Box – Directory Enumeration Labs – https://www.hackthebox.com